

MIP_PROMPT_LIBRARY_V2_PART4_GRAPH.md

Mainframe Intelligence Platform

Enterprise Prompt Library V2

Part 4 - Knowledge Graph, AST, Advanced Analysis, and Resilient Intelligence Prompts

Version: 3.0

Status: Strategic Enterprise Architecture

Purpose

The Knowledge Graph is the intelligence backbone of MIP.

Most modernization tools stop at inventories.

Some stop at metadata.

A few build dependency maps.

MIP goes significantly further.

Inventory

↓

Parsing

↓

AST Construction

↓

Semantic Extraction

↓

Metadata

↓

Relationships



Knowledge Graph



Control Flow Analysis



Data Flow Analysis



Graph Analytics



Reasoning



Modernization Intelligence



Transformation Intelligence



Business Intelligence



Operational Intelligence



Decision Intelligence

The objective is not simply to catalog assets.

The objective is to understand how the enterprise actually operates.

The graph enables engineers, architects, modernization teams, auditors, operations teams, and business stakeholders to answer questions that are nearly impossible to answer manually.

Examples:

- What breaks if I change this field?
 - Which programs update this table?
 - Which batch jobs consume this dataset?
 - What is the true root program?
 - Which components belong together?
 - Which business capability does this program support?
 - Which applications implement a specific business process?
 - Which APIs can be generated from existing transactions?
 - Which programs can be modernized together?
 - Which assets are dead code?
 - Which assets are modernization candidates?
 - Which systems are business critical?
 - Which dependencies create operational risk?
 - Which applications should be decomposed first?
 - Which assets represent technical debt?
 - Which business processes depend on a specific dataset?
 - Which interfaces are undocumented?
 - Which programs are unreachable?
 - Which applications have hidden coupling?
 - Which changes create regulatory impact?
-

Strategic Recommendation

Knowledge Graph Alone Is Not Enough

A pure metadata graph is valuable.

However, enterprise-scale modernization platforms require multiple complementary intelligence layers.

Recommended analysis stack:

1. Abstract Syntax Trees (AST)
2. Knowledge Graphs
3. Dependency Graphs
4. Data Flow Analysis
5. Control Flow Analysis
6. Program Slicing
7. Semantic Embeddings
8. LLM-Based Reasoning
9. Rule-Based Inference
10. Graph Algorithms
11. Business Capability Modeling
12. Application Boundary Detection
13. Event Flow Analysis
14. Runtime Telemetry Correlation
15. Operational Dependency Analysis

- 16. Risk Modeling
- 17. Change Impact Modeling
- 18. Architecture Discovery

The strongest architecture combines all of these approaches.

Recommended architecture:

Source Code



AST Layer



Semantic Extraction



Metadata Layer



Knowledge Graph



Graph Analytics



Inference Layer



LLM Reasoning



Modernization Intelligence



Transformation Intelligence

Why AST Matters

AST (Abstract Syntax Tree)

AST provides a structured representation of source code.

Instead of treating COBOL as text, AST represents executable intent.

Example:

```
PROGRAM
├─ DATA DIVISION
├─ WORKING STORAGE
├─ PROCEDURE DIVISION
│   └─ PARAGRAPH A
│       └─ IF STATEMENT
│           └─ PERFORM
│               └─ CALL
```

Benefits:

- Accurate dependency extraction
- Better impact analysis
- Dynamic call detection
- Data flow tracing
- Variable lineage
- Business rule extraction
- API generation support
- Automated refactoring support
- Dead code detection
- Reachability analysis
- Test impact analysis
- Rule mining
- Service extraction

Recommendation:

AST should become the canonical representation of:

- COBOL
- JCL
- Copybooks
- CICS
- IMS
- MQ
- DB2
- VSAM
- REXX

- Easytrieve
 - Scheduler Definitions
 - Control Cards
-

Additional Analysis Layers

Control Flow Graph (CFG)

Purpose

Model execution paths inside programs.

Supports:

- Dead code detection
- Reachability analysis
- Complexity analysis
- Refactoring recommendations
- Test coverage planning
- Risk assessment

Nodes:

- Statements
- Paragraphs
- Sections

Edges:

- EXECUTES_NEXT
 - BRANCHES_TO
 - RETURNS_TO
-

Data Flow Graph (DFG)

Purpose

Track movement of data through programs.

Supports:

- Field lineage
- Variable tracing
- Data transformation analysis
- Regulatory compliance
- Data governance
- Data quality analysis

Nodes:

- Variables
- Fields
- Tables
- Datasets

Edges:

- READS
 - WRITES
 - TRANSFORMS
 - DERIVES_FROM
-

Program Slicing

Purpose

Determine all code affected by a specific field, variable, table, dataset, transaction, or business rule.

Supports:

- Impact analysis
 - Change estimation
 - Test planning
 - Refactoring
 - Compliance reviews
-

Semantic Embeddings

Purpose

Understand business meaning beyond syntax.

Supports:

- Capability discovery
 - Duplicate functionality detection
 - Similar program clustering
 - Modernization candidate identification
 - Business process discovery
 - Service candidate identification
-

Event Flow Analysis

Purpose

Model asynchronous and event-driven execution.

Supports:

- MQ analysis
 - Trigger analysis
 - Event propagation
 - Hidden dependency discovery
-

Runtime Correlation Layer

Purpose

Correlate static analysis with operational evidence.

Supports:

- Execution frequency
 - Runtime criticality
 - Production dependency validation
 - Modernization prioritization
-

Graph Design Principles

Everything Is Connected

Programs do not exist in isolation.

Everything is modeled as relationships.

Relationships Are First-Class Citizens

The relationship is often more important than the node.

Explainability

Every graph edge must be traceable to source evidence.

Every recommendation must be explainable.

Every inference must be auditable.

Scalability

Graph must support:

- Millions of nodes
 - Tens of millions of edges
 - Continuous ingestion
 - Incremental updates
-

Multi-Layer Intelligence

The graph should not only store metadata.

It should connect:

- AST Nodes
 - Programs
 - Jobs
 - Transactions
 - Tables
 - Datasets
 - Business Capabilities
 - Applications
 - Domains
 - APIs
 - Services
 - Events
 - Interfaces
 - Business Processes
 - Users
 - Schedulers
 - Queues
-

Resilience

The graph must tolerate:

- Missing metadata
- Incomplete repositories
- Dynamic calls
- Undocumented interfaces
- Legacy naming conventions
- Partial source availability

Confidence scoring must be applied throughout the platform.

Evidence-Based Intelligence

Every relationship should include:

- Source Evidence
 - Discovery Method
 - Confidence Score
 - Validation Status
 - Timestamp
-

Recommended Graph Algorithms

The following algorithms should be incorporated into MIP.

Strongly Connected Components (SCC)

Purpose

Detect:

- Circular dependencies
- Program clusters
- Legacy application boundaries

Algorithms:

- Tarjan
 - Kosaraju
-

PageRank

Purpose

Identify critical programs.

Useful for:

- Modernization prioritization
 - Risk assessment
 - Critical asset discovery
-

Community Detection

Purpose

Automatically discover applications and domains.

Algorithms:

- Louvain
- Leiden

Outputs:

- Application clusters
 - Capability clusters
 - Domain clusters
-

Shortest Path Analysis

Purpose

Determine dependency paths.

Supports:

- Impact analysis
 - Root cause analysis
 - Dependency tracing
-

Centrality Analysis

Purpose

Identify highly connected assets.

Metrics:

- Degree Centrality
 - Betweenness Centrality
 - Eigenvector Centrality
-

Reachability Analysis

Purpose

Determine executable and non-executable assets.

Supports:

- Dead code detection
 - Root validation
 - Test planning
-

Graph Similarity Analysis

Purpose

Identify duplicate or overlapping functionality.

Supports:

- Rationalization
 - Consolidation
 - Service extraction
-

Influence Propagation Analysis

Purpose

Measure blast radius of changes.

Supports:

- Impact analysis
 - Risk estimation
 - Release planning
-

Prompt 16

Call Graph Construction

File

prompts/graph/build_call_graph.md

Purpose

Create a complete application call graph.

Context

Program-to-program relationships reveal execution architecture.

Inputs

Program Metadata

Relationship Metadata

AST Metadata

Runtime Evidence (Optional)

Instructions

Create graph.

Nodes:

- Programs
- Services
- Transactions

Edges:

- CALLS
- INVOKES
- EXECUTES

Capture:

- Direct Calls
- Recursive Calls
- Circular Dependencies
- Dynamic Calls
- External Service Calls
- MQ Invocations
- CICS Transfers
- IMS Calls

Use AST evidence wherever possible.

Generate traversal strategies.

Calculate:

- Reachability
- Centrality
- Criticality

Expected Output

Call Graph Design

Node Model

Edge Model

Traversal Examples

Critical Program Rankings

Dependency Hotspots

Constraints

Use evidence-based relationships only.

Success Criteria

Every program dependency represented.

Prompt 17

Batch Execution Graph

File

prompts/graph/build_batch_graph.md

Purpose

Model batch execution architecture.

Context

JCL jobs orchestrate business processing.

Inputs

Job Metadata

Program Metadata

Dataset Metadata

Scheduler Metadata

Control Card Metadata

Instructions

Create graph.

Nodes:

- Jobs
- Steps
- Programs
- Datasets
- Schedules
- Utilities

Edges:

- EXECUTES
- READS
- WRITES
- TRIGGERS
- DEPENDS_ON
- PRODUCES
- CONSUMES

Generate execution paths.

Identify:

- Critical Chains
- Failure Points
- Restart Dependencies

Expected Output

Batch Graph Architecture

Execution Traversals

Dependency Views

Critical Batch Chains

Operational Risk Analysis

Success Criteria

Batch execution chains become visible.

Prompt 18

Data Lineage Graph

File

prompts/graph/build_data_lineage_graph.md

Purpose

Construct enterprise data lineage.

Context

Data lineage is required for modernization, governance, compliance, and auditability.

Inputs

Dataset Metadata

DB2 Metadata

Program Metadata

AST Metadata

Data Flow Metadata

Instructions

Model:

- Producers
- Consumers
- Transformations

Track:

- Source Data
- Intermediate Data
- Target Data

Use AST and Data Flow Graph evidence.

Support:

- Dataset-Level Lineage
- Table-Level Lineage
- Field-Level Lineage

Expected Output

Lineage Model

Lineage Traversals

Impact Analysis Paths

Field-Level Lineage

Transformation Chains

Success Criteria

End-to-end lineage becomes traceable.

Prompt 19

Impact Analysis Graph

File

prompts/graph/impact_analysis.md

Purpose

Determine impact of proposed changes.

Context

Impact analysis is one of the highest-value modernization capabilities.

Inputs

Knowledge Graph

AST Metadata

Control Flow Graph

Data Flow Graph

Target Node

Instructions

Analyze:

- Upstream Dependencies
- Downstream Dependencies
- Consumers
- Producers
- Business Capabilities
- Applications
- Interfaces
- Data Flows
- Transactions

Generate impact report.

Calculate:

- Blast Radius
- Risk Score
- Confidence Score

Expected Output

Impact Summary

Affected Components

Affected Capabilities

Risk Assessment

Recommended Actions

Confidence Scores

Blast Radius Analysis

Success Criteria

Engineers can estimate change risk before implementation.

Prompt 20

Root Program Detection

File

prompts/graph/root_program_detection.md

Purpose

Identify true entry points.

Context

Many organizations do not know which programs are actual roots.

Inputs

Call Graph

Batch Graph

Transaction Metadata

Scheduler Metadata

CICS Metadata

MQ Metadata

IMS Metadata

Runtime Evidence

Instructions

Identify:

- Batch Drivers
- Online Drivers
- Scheduler Entry Points
- Utility Entry Points
- CICS Transaction Entry Points
- MQ Trigger Entry Points
- IMS Transaction Entry Points
- External Interface Entry Points
- API Entry Points

Classify roots by type.

Determine:

- Root Program
- Invocation Method
- Execution Frequency
- Downstream Reach
- Criticality Score
- Business Importance

Rank roots by business importance and execution impact.

Expected Output

Root Program Catalog

Entry Point Graph

Dependency Rankings

Critical Root Programs

Execution Reach Analysis

Business Criticality Rankings

Success Criteria

Execution entry points become visible.

All executable paths can be traced back to one or more root programs.

Business-critical roots are identified and prioritized.

Prompt 21

Business Capability Detection

File

prompts/graph/business_capability_detection.md

Purpose

Identify and map business capabilities implemented by the application portfolio.

Context

Modernization programs are increasingly organized around business capabilities rather than technical components.

Inputs

Program Metadata

Job Metadata

Dataset Metadata

DB2 Metadata

Transaction Metadata

Application Metadata

Knowledge Graph

Semantic Embeddings

Instructions

Identify logical business capabilities by analyzing:

- Program naming patterns
- Transaction names
- Dataset usage
- Table usage
- Batch processing flows
- Online transaction flows
- Shared business entities
- Existing documentation
- Semantic similarity

Group related assets into capabilities.

Generate capability hierarchies where appropriate.

Create relationships:

Nodes:

- Capability
- Application
- Program
- Job
- Dataset

- Table

Edges:

- IMPLEMENTS
- SUPPORTS
- USES
- OWNS
- DEPENDS_ON

Generate confidence scores.

Expected Output

Business Capability Catalog

Capability-to-Application Mapping

Capability Dependency Graph

Capability Confidence Scores

Business Domain Groupings

Constraints

All capability assignments must include supporting evidence.

Low-confidence assignments must be flagged for review.

Success Criteria

Technical assets can be traced to business capabilities.

Applications can be grouped by business function.

Modernization planning can be organized around business outcomes.

Prompt 22

Application Boundary Detection

File

prompts/graph/application_boundary_detection.md

Purpose

Automatically discover logical applications and domains.

Context

Many enterprises do not have accurate application inventories.

Inputs

Knowledge Graph

Call Graph

Batch Graph

Data Lineage Graph

Instructions

Apply:

- Community Detection
- SCC Analysis
- Dependency Clustering
- Semantic Clustering

Identify:

- Applications
- Domains
- Shared Services
- Integration Hubs

Generate confidence scores.

Expected Output

Application Catalog

Domain Catalog

Dependency Clusters

Shared Service Inventory

Application Confidence Scores

Success Criteria

Logical application boundaries become visible.

Prompt 23

Critical Asset Discovery

File

prompts/graph/critical_asset_discovery.md

Purpose

Identify assets that represent the highest business and technical risk.

Inputs

Knowledge Graph

Call Graph

Batch Graph

Lineage Graph

Runtime Evidence

Instructions

Analyze:

- Centrality
- Reachability
- Execution Frequency
- Business Capability Mapping
- Dependency Density

Rank:

- Programs
- Jobs
- Tables
- Datasets
- Transactions

Expected Output

Critical Asset Catalog

Risk Rankings

Success Criteria

Business-critical assets are objectively identified.

Prompt 24

Dead Code and Orphan Asset Detection

File

prompts/graph/dead_code_detection.md

Purpose

Identify unused, unreachable, and orphaned assets.

Inputs

Call Graph

Batch Graph

Root Program Catalog

Runtime Evidence

Instructions

Detect:

- Unreachable Programs
- Unused Copybooks
- Orphan Tables
- Orphan Datasets
- Dormant Transactions

Assign confidence scores.

Expected Output

Dead Code Inventory

Orphan Asset Catalog

Retirement Candidates

Success Criteria

Unused assets become visible and measurable.

Graph Completion Criteria

MIP should now answer:

- Which program calls this program?
- Which programs are impacted?
- Which jobs execute this program?
- Which datasets flow through the system?
- Which tables are affected?
- What are the root programs?
- Which business capability owns this program?
- Which applications support this capability?
- What business processes are impacted by a change?
- Which applications exist in the portfolio?
- Which assets are dead code?
- Which assets are modernization candidates?
- Which assets are business critical?
- Which dependencies create operational risk?
- Which interfaces are undocumented?
- Which applications should be decomposed first?
- What is the execution architecture?
- What is the blast radius of a change?
- Which assets can be retired safely?
- Which services can be extracted?
- Which APIs can be generated?
- Which domains exist within the enterprise?

At enterprise scale, the strongest approach is not Graph versus AST.

It is:

AST + Control Flow + Data Flow + Knowledge Graph + Graph Algorithms + Semantic Intelligence + Runtime Evidence + LLM Reasoning.

This combination is significantly more powerful than any individual technique and represents the architecture used by the most advanced modernization, architecture intelligence, and code intelligence platforms.

End of Part 4.